
UNIT IV - PL/SQL PROGRAMMING

4.5 Procedures:

Advantages, Creating, Executing and Deleting a Stored Procedure

Prepared By
Mr. Mohammad Ali
Lecturer (Selection Grade)
M. H. Saboo Siddik Polytechnic, Mumbai

Subprogram

- A **subprogram** is a program unit/module that performs a particular task.
- These subprograms are combined to form larger programs. This is basically called the '**Modular design**'.
- A subprogram can be invoked by another subprogram or program which is called the calling program.
- A subprogram can be created:
 - At schema level
 - Inside a package
 - Inside a PL/SQL block
- A **schema level subprogram** is a standalone subprogram. It is created with the **CREATE PROCEDURE** or **CREATE FUNCTION** statement. It is stored in the database and can be deleted with the **DROP PROCEDURE** or **DROP FUNCTION** statement.

Subprogram

- A subprogram created inside a package is a **packaged subprogram**. It is stored in the database and can be deleted only when the package is deleted with the **DROP PACKAGE** statement.
- **PL/SQL subprograms** are named PL/SQL blocks that can be invoked with a set of parameters. PL/SQL provides two kinds of subprograms:
 - **Functions**: these subprograms return a single value, mainly used to compute and return a value.
 - **Procedures**: these subprograms do not return a value directly, mainly used to perform an action.

Parts of a PL/SQL Subprogram(Procedures / Functions)

- Each PL/SQL subprogram has a name, and may have a parameter list. Stored Procedure or function's block of code in PL/SQL is made up of the following three sections:

Sr. No.	Parts & Description
1	Declarative Part / Section It is an optional part. However, the declarative part for a subprogram does not start with the DECLARE keyword. In this section, variables, constants, cursor or exceptions that are going to be used by procedure or function are declared. These items are local to the subprogram.
2	Executable Part / Section This is a mandatory part and contains statements that perform the designated action.
3	Exception-handling This is again an optional part. It contains the code that handles run-time errors.

Stored Procedure

1. A **Stored Procedure** is a type of code in SQL that can be stored for later use and can be used many times.
2. **Stored Procedures** are created to perform one or more DML operations on Database.
3. It is nothing but the group of SQL statements that accepts some input in the form of parameters and performs some task and **may** or **may not returns a value**.
4. A stored procedure or in simple a proc is a named PL/SQL block which performs one or more specific task. This is similar to a procedure in other programming languages.
5. A procedure has a header and a body. The header consists of the name of the procedure and the parameters or variables passed to the procedure. The body consists of declaration section, execution section and exception section similar to a general PL/SQL Block.
6. A procedure is similar to an anonymous PL/SQL Block but it is named for repeated usage.

Stored Procedure - Advantages

1. **Better Performance** –

- Compilation is automatically done by oracle engine.
- Whenever the calling of procedure or function is done ,the oracle engine loads the compiled code into a memory area called System Global Area(SGA) due to which execution becomes faster.

2. **Higher Productivity – Provides Reusability and avoids redundancy**

- Same block of code for procedure or function can be called any number of times for working on multiple data.
- Due to which number of lines of code cannot be written repeatedly. It results in higher productivity.

3. **Scalability** – Stored procedures increase scalability by isolating application processing on the server.

4. **Maintainability** – Maintaining a procedure on a server is much easier than maintaining copies on various client machines, this is because scripts are in one location.

5. **Maintains Security** – Access to the Oracle data can be restricted by allowing users to manipulate the data only through stored procedures that execute with their granted privileges.

6. **Saves Memory** – Stored procedure or function have shared memory. Due to which it saves memory as a single copy of either a procedure or a function can be loaded for execution by any number of users who have access permission.

7. **Maintains Integrity** – Integrity means accuracy. Use of procedure or function ensures integrity because they are stored as database objects by the oracle engine.

Difference between Stored Procedure and Function

Stored Procedure	Function
May or may not returns a value to the calling part of program.	Returns a value to the calling part of the program.
Uses IN, OUT, IN OUT parameter.	Uses only IN parameter.
Returns a value using “ OUT” parameter.	Returns a value using “RETURN”.
Does not specify the datatype of the value if it is going to return after a calling made to it.	Necessarily specifies the datatype of the value which it is going to return after a calling made to it.
Cannot be called from the function block of code.	Can be called from the procedure block of code.

Creating a Stored Procedure or Function

- Whenever a block of code for stored procedure or function is written it is then, they are **automatically compiled by the oracle engine**.
- During compilation **if any error** occurs, we get a message on the screen saying that the procedure or function is created with compilation errors but actual error is not displayed.
- In order to find out the compilation errors following statement can be executed:

SELECT * from USER_ERRORS;

- Once it is compiled, it is then stored by the oracle engine in the database as a **database object**.

Creating Stored Procedure - Syntax

- A procedure is created with the CREATE OR REPLACE PROCEDURE statement.
- The simplified syntax for the CREATE OR REPLACE PROCEDURE statement is as follows:

```
CREATE [OR REPLACE] PROCEDURE <procedure_name>
(<variable_name> IN/OUT/IN OUT <datatype>,
<variable_name> IN/OUT/IN OUT <datatype>,...) IS/AS
variable/constant declaration;
BEGIN
    -- PL/SQL subprogram body;
EXCEPTION
    -- Exception Handling block ;
END <procedure_name>;
```

Creating Stored Procedure - Syntax

Where,

- *procedure-name* specifies the name of the procedure.
- *variable_name* is the variable name for variable used in the stored procedure.
- **CREATE** is a keyword used for specifying the name of the procedure to be created.
- **[OR REPLACE]** option allows modifying an existing procedure.
- The **AS** keyword is used instead of the **IS** keyword for creating a standalone procedure.
- **IN** represents that value will be passed from outside and **OUT** represents that this parameter will be used to return a value outside of the procedure.
- **BEGIN**, **EXCEPTION** and **END** are keywords used to indicate different sections of the procedure being created.
- **PL/SQL subprogram body(procedure-body)** contains the executable part.
- At the end, *<procedure_name>* is optional to write, we can simply use **END** statement to end the procedure definition.

Creating Stored Procedure - Syntax

The most important part is parameters. Parameters are used to pass values to the Procedure. There are 3 different types of parameters, they are as follows:

1. *IN: refers to READ ONLY mode*

This is the Default Parameter for the procedure. It always receives the values from calling program or user.

2. *OUT: refers to WRITE ONLY mode*

This parameter always returns the values to the calling program.

3. *IN OUT: refers to READ AND WRITE mode*

This parameter performs both the operations. It Receives value from as well as sends the values to the calling program.

Example:

Creating Procedure to update Salary of a particular Employee with 1000

```
CREATE or REPLACE PROCEDURE INC_SAL(id IN NUMBER, up_sal OUT NUMBER) IS
BEGIN
UPDATE emp SET sal = sal+1000 WHERE empno = id;
COMMIT;
SELECT sal INTO up_sal FROM emp WHERE empno = id;
dbms_output.put_line('Increased salary of ' || id || ' = ' || up_sal);
END;
/
```

Steps to execute the procedure:

- Declare a Variable to Store the value coming out from Procedure :
VARIABLE v_sal NUMBER;
- Execution of the Procedure:
EXECUTE INC_SAL(7900, :v_sal);
- To check the updated salary use SELECT statement:
SELECT * FROM emp WHERE empno = 7900;
- or Use print statement :
PRINT :v_sal

Example:

Executing Procedure to update Salary of empno=7900 with 1000

```
SQL> select * from emp where empno=7900;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7900	JAMES	CLERK	7698	03-DEC-81	950		30

```
SQL> variable v_sal number;
```

```
SQL> exec inc_sal(7900, :v_sal);
```

```
Increased salary of 7900= 1950
```

```
PL/SQL procedure successfully completed.
```

```
SQL> select * from emp where empno=7900;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7900	JAMES	CLERK	7698	03-DEC-81	1950		30

```
SQL> print :v_sal;
```

```
V_SAL
-----
1950
```

```
SQL> _
```

Example: Creating and executing Procedure to add two numbers

```
CREATE OR REPLACE PROCEDURE Sum(a IN number, b IN number) IS
c number;
BEGIN
    c := a+b;
    dbms_output.put_line('Sum of two nos= '|| c);
END Sum;
/
```

OUTPUT:

Procedure created.

Executing the Procedure through PL/SQL code :

```
set verify off;
DECLARE
    x number;
    y number;
BEGIN
    x := &x;
    y := &y;
    Sum(x,y);
END;
/
```

OUTPUT:

```
Enter value for x: 10
Enter value for y: 30
Sum of two nos= 40

PL/SQL procedure successfully completed.
```

Executing the procedure directly by providing two parameters on SQL prompt:

```
SQL> exec sum(10,20);
Sum of two nos= 30

PL/SQL procedure successfully completed.
```

Example: Creating and executing Procedure to add two numbers

```
CREATE OR REPLACE PROCEDURE Sum IS
a number :=20;
b number :=40;
c number;
BEGIN
    c := a+b;
    dbms_output.put_line('Sum of two nos= '|| c);
END Sum;
/
```

OUTPUT:

Procedure created.

Executing the procedure directly without providing parameters on SQL prompt:

```
SQL> exec sum;
Sum of two nos= 60

PL/SQL procedure successfully completed.
```

Example:

- The following example creates a simple procedure that displays the string 'Hello World!' on the screen when executed.

```
CREATE OR REPLACE PROCEDURE greetings AS
```

```
BEGIN
```

```
    dbms_output.put_line('Hello World!');
```

```
END;
```

```
/
```

- When above code is executed using SQL prompt, it will produce the following result:

Procedure created.

Executing a Standalone Procedure

- A standalone procedure can be called in two ways:
 - Using the EXECUTE keyword
 - Calling the name of the procedure from a PL/SQL block
- The above procedure named 'greetings' can be called with the EXECUTE keyword as:

EXECUTE greetings;

- The above call would display:

Hello World!

PL/SQL procedure successfully completed.

- The procedure can also be called from another PL/SQL block:

BEGIN

greetings;

END;

/

- The above call would display:

Hello World!

PL/SQL procedure successfully completed.

Deleting a Standalone Procedure

- A standalone procedure is deleted with the **DROP PROCEDURE** statement.
- Syntax for deleting a procedure is:
DROP PROCEDURE procedure-name;
- So you can drop *greetings* procedure by using the following statement:

DROP PROCEDURE greetings;